



# Formación

## Curso Angular 20

indra



## Formador

### Ana Isabel Vegas



INGENIERA INFORMÁTICA con Master Máster Universitario en Gestión y Análisis de Grandes Volúmenes de Datos: Big Data, tiene la certificación PCEP en Lenguaje de Programación Python y la certificación JSE en Javascript. Además de las certificaciones SCJP Sun Certified Programmer for the Java 2 Platform Standard Edition, SCWD Sun Certified Web Component Developer for J2EE 5, SCBCD Sun Certified Business Component Developer for J2EE 5, SCEA Sun Certified Enterprise Architect for J2EE 5.

Desarrolladora de Aplicaciones FULLSTACK, se dedica desde hace + de 20 años a la CONSULTORÍA y FORMACIÓN en tecnologías del área de DESARROLLO y PROGRAMACIÓN.



[training@iconotc.com](mailto:training@iconotc.com)

❑ **Duración:** 35 horas

❑ **Modalidad:** On-line

❑ **Fechas/Horario:**

- Días 6, 7, 8, 9 y 10 Julio 2026
- Horario De L a V de 8:00 – 15:00 hs

❑ **Contenidos:**

- Introducción a Angular y preparación del entorno

¿Qué es Angular y qué resuelve?

Angular: fundamentos clave

Angular vs otros frameworks

Preparar el entorno

Crear el primer proyecto Angular

- Componentes en profundidad

Declaración de Componentes y Standalone Components

Comunicación entre componentes con @Input y @Output

Ciclo de vida del componente

Estilos: encapsulados y globales

- Templates, directivas y pipes

Binding de datos en plantillas

Directivas estructurales

Directivas de atributo

Pipes integrados

Creación de pipes personalizados

- Enrutamiento con provideRouter

Fundamentos del routing en Angular 20

Configuración de rutas con provideRouter y Route

Navegación entre vistas

Parámetros de ruta y query params

Lazy loading moderno

Guards básicos

Caso práctico: aplicación de un restaurante

- Servicios e inyección de dependencias

Introducción a los servicios

Creación e inyección de servicios

Ámbito y ciclo de vida de servicios

Comunicación HTTP con HttpClient

Manejo básico de errores en peticiones HTTP

Interceptors HTTP para manejo global de peticiones y errores

Lazy services y carga perezosa de servicios

- Formularios template-driven

Estructura básica de formularios template-driven

Binding y sincronización con ngModel

Validaciones básicas en formularios template-driven

Mostrar mensajes de error

Formularios anidados simples

- Formularios reactivos

Introducción a formularios reactivos

Validaciones reactivas

Manejo programático de errores

Validadores personalizados

Validación de formularios complejos

- Signals y estado reactivo

Introducción a Signals

API básica de Signals

Estado local reactivo sin necesidad de Observables

Uso de Signals en templates Angular

Diferencias entre Signals y RxJS

Integración y coexistencia entre Signals y RxJS

- Comunicación avanzada y manejo de estado

Comunicación entre componentes hermanos

Comparativa: EventEmitter vs Subject

Patrones de arquitectura recomendados para manejo de estado

Uso básico de BehaviorSubject y Signals compartidos

Introducción a estado compartido simple sin NgRx

Conceptos de gestión de estado reactivo a mayor escala (sin librerías externas)

- Testing de componentes y servicios

Importancia de testear en Angular

Configuración del entorno de testing

Pruebas de componentes

Pruebas de servicios

Pruebas de formularios

Introducción a pruebas end-to-end (E2E) con Cypress o Playwright

- Arquitectura escalable y buenas prácticas

Organización por features (Feature Folders)

Reutilización de componentes y servicios

Nomenclatura, rutas y estructura limpia

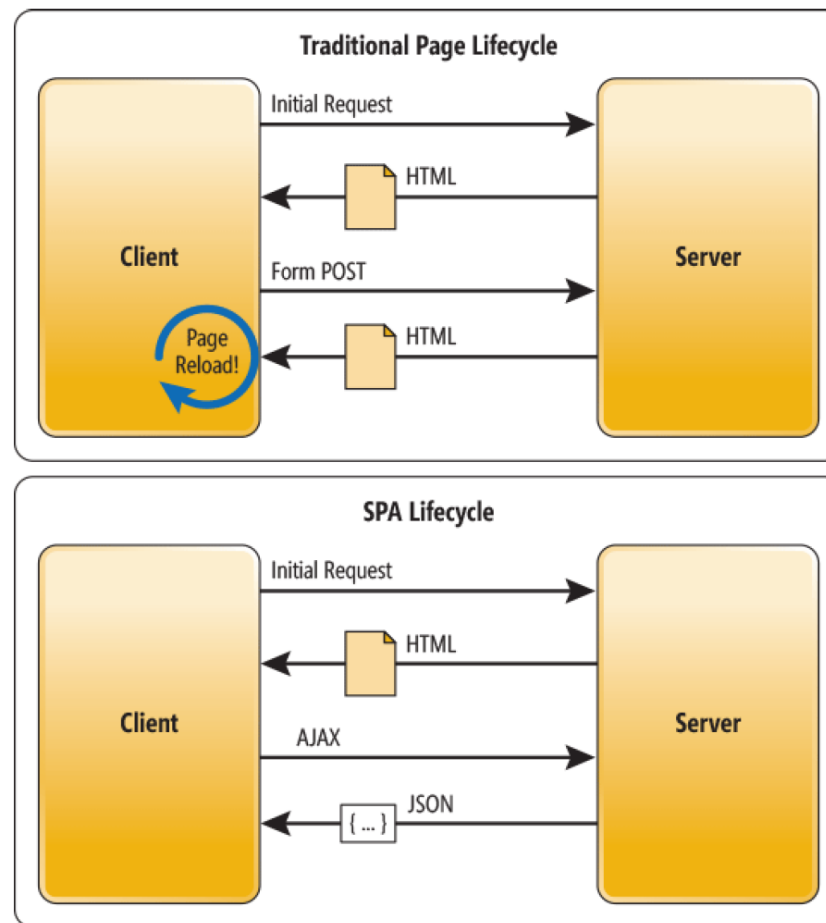
Introducción a NgModules

Migración y coexistencia entre NgModules y Standalone Components

# QUÉ ES ANGULAR?

- Básicamente es un Framework de JS que te permite crear Single Page Applications (o SPA's).
- Eso significa que con Angular nunca te va a abrir otra página. Todo será en la misma. Angular se encargará de ello. Esto lo hace de una forma muy elegante, simple y reactiva.

# SPA



# VERSIONES DE ANGULAR



AngularJS

1.0 , 1.1 , 1.2 .. 1.7



Angular

2.0 , 4.0 , 5.0 6.0,  
7.0...



# PUESTA A PUNTO DEL ENTORNO

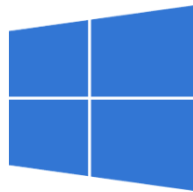
- Las herramientas que usaremos a lo largo del curso son:
  - **Navegador web** - Chrome es el que usaremos pero puedes usar el que te parezca mejor.
  - **Editor de texto** - Visual Studio Code, PHP Storm, Atom,, ...etc.
  - **Node.js** específicamente NPM.
  - **Angular CLI**, es un set de herramientas para la consola que nos facilita el trabajo con Angular.



# VISUAL STUDIO CODE

- Se puede descargar de esta url:

<https://code.visualstudio.com/download>



↓ Windows

Windows 7, 8, 10

|                  |        |        |
|------------------|--------|--------|
| User Installer   | 64 bit | 32 bit |
| System Installer | 64 bit | 32 bit |
| .zip             | 64 bit | 32 bit |



↓ .deb

Debian, Ubuntu

↓ .rpm

Red Hat, Fedora, SUSE

|         |        |        |
|---------|--------|--------|
| .deb    | 64 bit | 32 bit |
| .rpm    | 64 bit | 32 bit |
| .tar.gz | 64 bit | 32 bit |

[Snap Store](#)



↓ Mac

macOS 10.9+

# NODE.JS

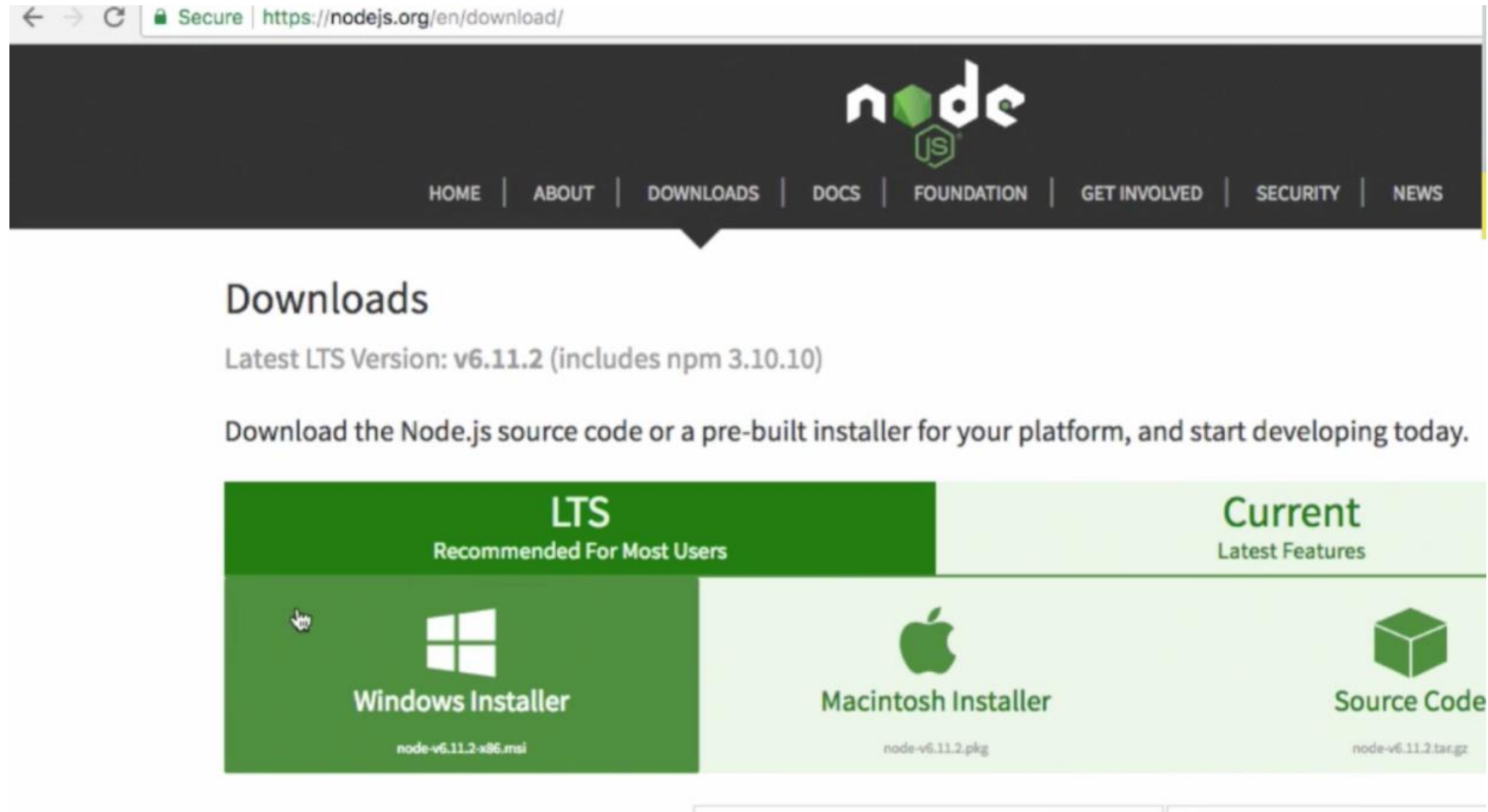
- Node.js, en la consola validamos que tenemos instalado npm con el comando:

```
npm -v
```

- De no ser así nos lo descargamos de la página oficial:

<https://nodejs.org/en/download/prebuilt-installer>

# NODE.JS



# ANGULAR CLI

- Recuerda ejecutar los comandos con privilegios de administrador.

```
npm config set proxy http://10.0.8.102:8080
```

```
npm config rm proxy
```

```
npm install -g @angular/cli@20
```

- Si habíamos instalado una versión de Angular-CLI previamente y queremos actualizarla, debemos ejecutar los siguientes comandos:

```
1. npm uninstall -g @angular/cli
```

```
2. npm cache clean --force
```

```
3. npm install -g @angular/cli@20
```

# NUESTRO PRIMER PROYECTO



The image shows a file explorer for an Angular 4 project named 'curso-angular4'. The file list includes folders like 'e2e', 'node\_modules', and 'src', and files like 'app.component.css', 'app.component.html', 'app.component.spec.ts', 'app.component.ts', 'app.module.ts', 'assets', 'environments', 'favicon.ico', 'index.html', 'main.ts', 'polyfills.ts', 'styles.css', 'test.ts', 'tsconfig.app.json', 'tsconfig.spec.json', 'typings.d.ts', '.angular-cli.json', '.editorconfig', '.gitignore', 'curso-angular4.sublime-project', 'curso-angular4.sublime-workspace', 'karma.conf.js', 'package.json', 'protractor.conf.js', 'README.md', 'tsconfig.json', and 'tslint.json'.

**Archivos estáticos (imágenes de la aplicación)**

**Estilos globales de la aplicación**

- **.ts**: type script
- **app.component.ts**  
Componente principal de la aplicación donde se define la clase del componente (AppComponent):
- **.html** es la plantilla del componente
- **.css** especifica la presentación/vista del componente

Podemos aplicar MVC para la estructuración de las carpetas del proyecto Angular.

**package.json**: el archivo de configuración de un proyecto de Node. Contiene las dependencias/librerías/paquetes de Angular que vamos a utilizar en nuestra aplicación y que Angular CLI (via NPM) se ha encargado de instalarlas por nosotros en la carpeta `node_modules`. Estas son las librerías imprescindibles para nuestro proyecto y podremos seguir añadiendo nuevas librerías para cubrir las funcionalidades y requisitos del proyecto.

**index.html**: es dónde se va a *renderizar* la aplicación

# ELIMINAR NODE-MODULES

- Si el proyecto pesa demasiado podemos eliminar la carpeta node-modules.
- Nos dara errores de compilacion y no podremos ejecutar el proyecto.
- Para volver a descargar todas las dependencias:
  1. Nos posicionamos dentro del proyecto en la consola
  2. Ejecutamos: `npm install` (`npm i`)

# COMPONENTES EN ANGULAR

- Un módulo en un app, tiene una entidad propia y puede estar compuesto por más cosas como componentes.
- Cada vista debería ser un componente, y está puede o no, componerse por otros componentes, dependiendo de la complejidad, profundidad y la reutilización de código.



# COMPONENTES EN ANGULAR

- Categorías:
  - Imports: módulos externos a importar.
  - Providers: servicios necesarios.
  - Bootstrap: con que componente vamos a iniciar.
- @Component: Palabra reservada para declarar un componente, contiene varios elementos:
  - selector: Tag de HTML para insertar el componente
  - templateUrl: Archivo HTML que va a usar el componente
  - styleUrls: Archivos CSS que usará el template

# COMPONENTES EN ANGULAR

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})
```

# CICLO DE VIDA DE UN COMPONENTE

- El ciclo de vida es el conjunto de eventos que Angular ejecuta durante la existencia de un componente:
  1. Creación
  2. Inicialización
  3. Detección de cambios
  4. Renderizado
  5. Destrucción
- Angular permite reaccionar a estos momentos mediante métodos especiales llamados **hooks** del ciclo de vida.

# CICLO DE VIDA DE UN COMPONENTE

- Orden de ejecución de los hooks

```
constructor()  
ngOnChanges()  
ngOnInit()  
ngDoCheck()  
ngAfterContentInit()  
ngAfterContentChecked()  
ngAfterViewInit()  
ngAfterViewChecked()  
  
(ngDoCheck, ngAfterContentChecked y ngAfterViewChecked  
se ejecutan repetidamente durante la detección de cambios)  
  
ngOnDestroy()
```

# CICLO DE VIDA DE UN COMPONENTE

- Constructor:
  - No es un hook de Angular sino un método de TypeScript.
  - Se ejecuta cuando Angular crea la instancia del componente.
  - Se recomienda para DI e inicializaciones simples.
- `ngOnChanges()`
  - Se ejecuta cuando cambia el valor de una propiedad decorada con `@Input()`.
- `ngOnInit()`
  - Se ejecuta una única vez después de la primera asignación de los `@Input()`.
  - Se recomienda para llamadas a APIs, Carga inicial de datos, ... etc.

# CICLO DE VIDA DE UN COMPONENTE

- `ngDoCheck()`
  - Se ejecuta en cada ciclo de detección de cambios.
- `ngAfterContentInit()`
  - Se ejecuta una vez después de que Angular proyecte contenido mediante `<ng-content>`.
- `ngAfterContentChecked()`
  - Se ejecuta después de cada verificación del contenido proyectado.
- `ngAfterViewInit()`
  - Se ejecuta una vez cuando la vista del componente y sus hijos están completamente inicializados.

# CICLO DE VIDA DE UN COMPONENTE

- `ngAfterViewChecked()`
  - Se ejecuta después de cada verificación de la vista.
- `ngOnDestroy()`
  - Se ejecuta justo antes de destruir el componente.

# CICLO DE VIDA DE UN COMPONENTE

Angular 20 ha introducido APIs adicionales que complementan los hooks clásicos:

- `afterNextRender()`
  - Se ejecuta después del siguiente renderizado.
- `afterEveryRender()`
  - Se ejecuta tras cada renderizado.
- `DestroyRef`
  - Alternativa moderna a `ngOnDestroy()` para tareas de limpieza.

```
import { DestroyRef, inject } from '@angular/core';

destroyRef = inject(DestroyRef);

constructor() {
  this.destroyRef.onDestroy(() => {
    console.log('Limpieza');
  });
}
```



# DATA BINDING

- Data binding es la comunicación entre tu código de TypeScript y el HTML. Siempre ten presente que al cliente le importa lo que ve, así que comunicar nuestra lógica (TypeScript) al template (HTML) es muy importante, para esto contamos con 4 tipos de Data Binding,

# TIPOS DE DATA BINDING EN ANGULAR

- **String Interpolation** `{{}}` TypeScript => HTML: Tener información (Variable, Array, por ejemplo) y presentarla a los usuarios en modo de HTML.
- **Property Binding** `[]` TypeScript <= HTML: Información del lado de HTML que puede ser por ejemplo, información que el usuario ingrese o que nosotros pongamos un valor por defecto. Viajando la información de HTML a TypeScript.
- **Event Binding** `()` TypeScript <= HTML: Escuchar eventos desde HTML y pasarlo a TypeScript.
- **Two Way Data Binding** `[()]` TypeScript <=> HTML: Comunicación de dos Vías. De lo que el cliente ve a TypeScript como de TypeScript hacia lo que el cliente ve.

# GESTIÓN DE ESTADO CON SIGNALS

- Las Signals son el sistema reactivo introducido en Angular para gestionar el estado de forma eficiente y declarativa.
- Características:
  - Reactividad nativa en Angular.
  - Actualización automática de la UI.
  - Menor dependencia de RxJS para estados locales.
  - Mejor rendimiento gracias a actualizaciones más precisas.

```
const contador = signal(0);
```

- Cuando el valor cambia, Angular actualiza automáticamente las vistas que lo utilizan.

# CREACIÓN Y ACTUALIZACIÓN DE SIGNALS

- Crear Signals :

```
import { signal } from '@angular/core';  
  
contador = signal(0);
```

- Leer el valor:

```
contador()
```

- Actualizar el valor:

```
contador.set(10);
```

- Modificar a partir del valor actual:

```
contador.update(valor => valor + 1);
```

# COMPUTED SIGNALS

- Una **computed signal** calcula valores automáticamente a partir de otras signals.

```
import { signal, computed } from '@angular/core';

precio = signal(100);

iva = signal(0.21);

precioFinal = computed(
  () => this.precio() * (1 + this.iva())
);
```

# EFFECTS CON SIGNALS

- Los **effects** ejecutan código cuando cambia una signal utilizada dentro de ellos.

```
import { effect } from '@angular/core';

constructor() {

  effect(() => {
    console.log(
      `Contador: ${this.contador()}`
    );
  });
}
```

# SIGNALS VS RXJS

| Signals           | RxJS                 |
|-------------------|----------------------|
| Estado local      | Flujos complejos     |
| Componentes       | WebSockets           |
| UI reactiva       | Streams asíncronos   |
| Fácil aprendizaje | Operadores avanzados |
| Menos código      | Mayor potencia       |

# COMANDOS NG

| Scaffold  | Usage                                        |
|-----------|----------------------------------------------|
| Component | <code>ng g component my-new-component</code> |
| Directive | <code>ng g directive my-new-directive</code> |
| Pipe      | <code>ng g pipe my-new-pipe</code>           |
| Service   | <code>ng g service my-new-service</code>     |
| Class     | <code>ng g class my-new-class</code>         |
| Guard     | <code>ng g guard my-new-guard</code>         |
| Interface | <code>ng g interface my-new-interface</code> |
| Enum      | <code>ng g enum my-new-enum</code>           |
| Module    | <code>ng g module my-module</code>           |



# DIRECTIVAS

- Las directivas son una forma elegante y rápida de manipular la información, contamos con 3 tipos de directivas:
  - COMPONENTES: Son directivas que siempre tienen asignados templates de HTML.
  - ESTRUCTURALES: Nos permiten modificar el DOM, es decir manipular elementos de HTML.
  - ATRIBUTOS: A estas directivas les indicamos a través de HTML cómo se deben comportar de acuerdo con ciertas condiciones que definimos.

# DIRECTIVA NGFOR

- Es una directiva estructural. Nos permite iterar sobre una colección.

```
<ul>
  <li *ngFor="let hero of heroes">{{hero.name}}</li>
</ul>
```

```
@for (item of items; track item.name) {
  <li>{{ item.name }}</li>
} @empty {
  <li>There are no items.</li>
}
```

# DIRECTIVA NGIF

- Directiva ngIf es una directiva estructural, ésta directiva nos permite mostrar elementos de HTML de acuerdo a una condición que definamos.

```
<div *ngIf="hero" class="name">{{hero.name}}</div>
```

```
@if (a > b) {  
    {{a}} is greater than {{b}}  
} @else if (b > a) {  
    {{a}} is less than {{b}}  
} @else {  
    {{a}} is equal to {{b}}  
}
```

# DIRECTIVA NGSTYLE

- Es una directiva de atributo por eso va entre corchetes.
- ngStyle se utiliza para aplicar un estilo CSS en función de una condición:

```
<some-element [ngStyle]="{'font-style': styleExp}">...</some-element>
```

```
<some-element [ngStyle]="{'max-width.px': widthExp}">...</some-element>
```

```
<some-element [ngStyle]="objExp">...</some-element>
```

# DIRECTIVA NGCLASS

- Es otra directiva de atributo.
- ngClass se utiliza para aplicar una clase de estilo definida en un fichero css.

```
<some-element [ngClass]='''first second''>...</some-element>
```

```
<some-element [ngClass]='["'first', 'second']">...</some-element>
```

```
<some-element [ngClass]='{"'first': true, 'second': true, 'third': false}">...</some-element>
```

```
<some-element [ngClass]="stringExp|arrayExp|objExp">...</some-element>
```

```
<some-element [ngClass]='{"'class1 class2 class3' : true}">...</some-element>
```

# DIRECTIVAS NGSWITCH / NGSWITCHCASE

- ngSwitch -> directiva de atributo
- ngSwitchCase -> directiva estructural

```
<div [ngSwitch]="hero?.emotion">
  <app-happy-hero      *ngSwitchCase="'happy'"      [hero]="hero"></app-happy-hero>
  <app-sad-hero        *ngSwitchCase="'sad'"          [hero]="hero"></app-sad-hero>
  <app-confused-hero   *ngSwitchCase="'app-confused'" [hero]="hero"></app-confused-hero>
  <app-unknown-hero    *ngSwitchDefault              [hero]="hero"></app-unknown-hero>
</div>
```

```
@switch (condition) {
  @case (caseA) {
    Case A.
  }
  @case (caseB) {
    Case B.
  }
  @default {
    Default case.
  }
}
```

# DIRECTIVAS PERSONALIZADAS

- Podemos utilizar las directivas propias de Angular o crear las nuestras propias.
- Una buena practica es crear una carpeta directivas para tal fin.
- En dicha carpeta creamos un archivo typescript donde definimos la directiva.

# DIRECTIVAS PERSONALIZADAS

```
@Directive({
```

```
    selector: 'nombre a utilizar en el html'
```

```
})
```

```
export class {
```

```
    // implementacion de la directiva
```

```
}
```



# HOSTLISTENER

- Los *host listeners* nos ayudan a escuchar los eventos de manera muy sencilla, tiene tres parámetros, el evento, el objetivo del evento y el código a ejecutar cuando se ejecute el evento.

# HOSTBINDERS

- Host Binding nos permite editar elementos del DOM o el HTML al que esté asignado desde nuestra directiva.
- Recibe el atributo del HTML que queremos modificar.

# ROUTER

- El *routing* en Angular nos permite implementar navegación en nuestra aplicación, podemos asignar vistas/componentes específicos para cada url que deseemos.
- Routing nos permite enviar y recibir parámetros.
- En app.config.ts hay que importar el proveedor de rutas:

```
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideBrowserGlobalErrorListeners(),  
    provideZoneChangeDetection({ eventCoalescing: true }),  
    provideRouter(routes)  
  ]  
};
```

# ROUTER

- En app.routes.ts se mapean todas las rutas de la aplicación:

```
const misRutas: Routes = [  
  { path: 'us', component: NosotrosComponent },  
  { path: 'contact', component: ContactoComponent },  
  { path: 'tab', component: TablaComponent },  
  { path: 'index', component: HomeComponent },  
  { path: '', redirectTo: 'index', pathMatch: 'full' }  
];
```

# ROUTERMODULE

- Para poder agregar un nuevo componente que va a tener su propia ruta asignada necesito el modulo de RouterModule
- Importo RouterModule en app.routes.ts :

```
// Importar modulo RouterModule  
import { Routes, RouterModule } from '@angular/router';
```

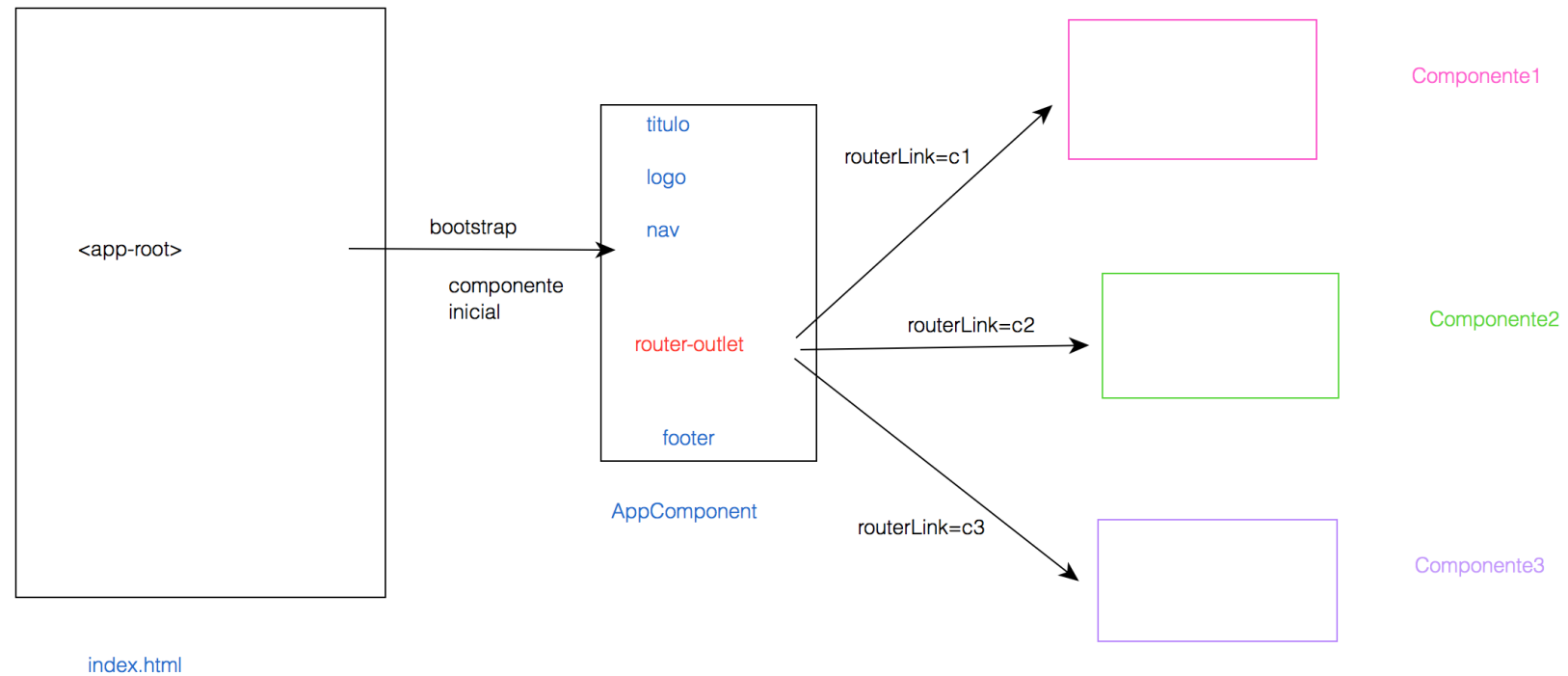
# ROUTERLINK

- Para implementar navegación en SPA's la mejor forma de hacerlo es usando el *routerLink* que nos provee Angular ya que así reducimos los tiempos de carga y aprovechamos al máximo las capacidades del *framework*.

# HREF Y ROUTERLINK

- Angular es ideal para SPA (Single Page Application), la diferencia esta en:
  - **href** -> enlace tradicional, con redirección, si se hace click te lleva a otra página (como si fuera otra distinta), recargando todo, no recomendable para una SPA.
  - **routerLink** -> mantiene el index.html y usando `<router-outlet></router-outlet>` quita y pone el componente al que hace referencia, mostrándolo en esa parte del index.html.

# EJEMPLO ROUTER





# LINK ACTIVO

- El router nos permite resaltar el link que se encuentra activo.
- Puedes usar routerLinkActive="cualquier clase", le puede indicar a angular la clase que hace que se resalte el link que están visitando, para este caso bootstrap usa la clase active

```
<li class="nav-item">  
  <a class="nav-link" routerLink="detalle/3"  
    routerLinkActive="active">Detalle</a>  
</li>  
<li class="nav-item">  
  <a class="nav-link" routerLink="contacto"  
    routerLinkActive="active">Contacto</a>  
</li>
```

# PARÁMETROS EN RUTAS

- Ruta/:parametro

```
const misRutas: Routes = [  
  {path: 'inicio', component: AppComponent},  
  {path: 'contacto', component: ContactoComponent},  
  {path: 'catalogo', component: CatalogoComponent},  
  {path: 'detalle/:id', component: DetalleComponent}  
];
```

# PARÁMETROS TIPO QUERY

- Son parámetros opcionales que declaro en el enlace con la directiva queryParams.

```
<li class="nav-item">
  <a class="nav-link" routerLink="catalogo"
    [queryParams]= "{categoria:'zapatos',color:'negro'}"
    routerLinkActive="active">Catalogo</a>
</li>
```

# PARÁMETROS EN RUTAS

- Para recuperar los parámetros:

```
constructor(private route: ActivatedRoute) {  
  this.codigo = this.route.snapshot.params['id'];  
}
```

```
constructor(private route: ActivatedRoute) {  
  this.categoria = this.route.snapshot.queryParams['categoria'];  
  this.color = this.route.snapshot.queryParams['color'];  
}
```

# GUARDS

- Un Guard protege rutas y controla el acceso a ellas según condiciones.
- Tipos principales
  - CanActivate → decide si se puede acceder a una ruta.
  - CanDeactivate → controla si se puede salir de una ruta.
  - CanLoad → evita que un módulo sea cargado.
  - CanMatch → valida rutas con patrones.

# EJEMPLO DE GUARDS

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {

  constructor(private authService: AuthService,
               private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isLoggedIn()) {
      return true;
    } else {
      this.router.navigate(['/login']);
      return false;
    }
  }
}
```

```
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [AuthGuard]
}
```

# INTERCEPTORS

- Un Interceptor es un servicio que intercepta todas las solicitudes HTTP y puede modificar:
  - Cabeceras
  - Datos
  - Manejo de errores
  - Autenticación

# EJEMPLO DE INTERCEPTORS

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {

  intercept(req: HttpRequest<any>,
            next: HttpHandler): Observable<HttpEvent<any>> {

    const token = localStorage.getItem('token');

    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });

    return next.handle(authReq);
  }
}
```

```
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

En app.config.ts



# FORMULARIOS REACTIVOS

- Hay que importar los módulos **FormsModule** y **ReactiveFormsModule** en el componente

```
imports: [  
  BrowserModule,  
  AngularFireModule.initializeApp(environment.firebase),  
  FormsModule,  
  ReactiveFormsModule
```

# FORMULARIOS REACTIVOS

```
private fb = inject(NonNullableFormBuilder);

amigoForm = this.fb.group({
  id: [0, [Validators.required, Validators.min(1)]],
  nombre: ['', [Validators.required, Validators.minLength(3)]],
  telefono: ['', [Validators.required, Validators.pattern('^([6-7][0-9]{8}$)')]
});
```

# FORMULARIOS REACTIVOS

```
<form [formGroup]="amigoForm">
  ID: <input type="text" formControlName="id" />
  @if (this.amigoForm.get('id')?.errors?.['required']) {
    <span>
      * Campo obligatorio
    </span>
  }
  @if (this.amigoForm.get('id')?.errors?.['min']) {
    <span>
      * Valor minimo 1
    </span>
  }
}
```

# SERVICIOS

- Los servicios nos permiten centralizar funcionalidad de nuestra aplicación, pueden ser usados desde cualquier componente y hacen nuestra aplicación mas mantenible.

# CREAR UN SERVICIO

- Desde linea de comandos podemos crear el servicio.
- Es importante poner el nombre de la carpeta para que no lo ubique en app.

```
[MacBook-Pro-de-Ana:MyGoogleMaps anaisabelvegascaceres$ ng g service services/negocios  
CREATE src/app/services/negocios.service.spec.ts (343 bytes)  
CREATE src/app/services/negocios.service.ts (137 bytes)
```

# DECLARACIÓN DEL SERVICIO

```
import { Injectable } from '@angular/core';
import { Negocio } from '../modelos/Negocio';

@Injectable({
  providedIn: 'root'
})
export class NegociosService {

  negocios: Negocio[] = [
    {id:1,nombre:'Bar Numancia',lat:40.4359819,
    {id:2,nombre:'Rodilla',lat:40.4335952 ,long
    {id:3,nombre:'Grupo Atrium',lat:40.437171 ,
    {id:4,nombre:'KutxaBank',lat:40.4358394 ,lo
    {id:5,nombre:'AulaCenter',lat:40.4367342 ,l
    {id:6,nombre:'Cafeteria Albahaca',lat:40.43

  ];

  public getNegocios(){
    return this.negocios;
  }
}
```

# INYECTAR EL SERVICIO

- El componente que desee hacer uso del servicio debe inyectarlo

```
private negociosService = inject(NegociosService);
```

# HTTP

- Importar el proveedor Http en app.config.ts:

```
import { provideHttpClient } from '@angular/common/http';
```



# HTTP

- En el servicio donde vamos a emitir la petición al servicio REST, hay que importar los recursos necesarios:

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
```

- Y declaramos la URL y las cabeceras:

```
url: "http://176.655.54.12:8080/miAplicacion/servicioREST";  
headers = new HttpHeaders({"Content-type": "application/json"});
```

# HTTP

- Peticiones a través de **HttpClient**:

```
constructor(private http: HttpClient) { }  
  
getAll(): Observable<any>{  
|   return this.http.get(this.url, {headers: this.cabeceras});  
}
```

# PIPES

- Los pipes son elementos que usamos en el DOM en el HTML junto con las directivas que van a tomar un elemento de entrada y le van a dar cierto formato y nos van a entregar una salida diferente.
- Angular trae por defecto una cantidad de pipes para configuraciones y cambios comunes. Por ejemplo, cambiar las letras de mayúsculas a minúsculas, formatear fechas, etc.
- Los pipes pueden tomar parámetros que le indiquemos.
- Es posible que encadenemos pipes hasta que obtengamos el resultado que deseamos.
- Además de los pipes que Angular tiene por defecto, es posible que nosotros hagamos nuevos.

# EJEMPLOS

- {{ texto | uppercase }}
- {{ fecha | date:'short' }}
- {{ importe | currency:'€ ':'code':'.1-1' }}

```
<!-- npm i @angular/localize
en angular.json incluir i18n con locale es

"projects": {
  "Ejemplo6-pipes": {
    "i18n":{
      "sourceLocale": "es"
    },
    "projectType": "application",
  }
}
-->
```

# TESTING EN ANGULAR

- **Por qué hacer pruebas?**
  - Detectar errores antes de producción.
  - Facilitar el mantenimiento del código.
  - Reducir regresiones al añadir nuevas funcionalidades.
  - Documentar el comportamiento esperado de la aplicación.
- **Tipos de pruebas**
  - Unitarias → funciones, servicios y componentes aislados.
  - Integración → interacción entre varios elementos.
  - End-to-End (E2E) → comportamiento completo de la aplicación.

# HERRAMIENTAS

- Angular utiliza principalmente:
  - **Jasmine** → definición de pruebas.
  - **Karma** o alternativas modernas como Vitest.
  - **Angular TestBed** → entorno de pruebas para componentes y servicios.

```
describe('Calculadora', () => {  
  
  it('debe sumar dos números', () => {  
  
    const resultado = 2 + 3;  
  
    expect(resultado).toBe(5);  
  
  });  
  
});
```

- describe() → agrupa pruebas.
- it() → caso de prueba.
- expect() → resultado esperado.

# TESTING DE FUNCIONES

- Función simple:

```
export function multiplicar(a: number, b: number): number {  
  return a * b;  
}
```

- Test:

```
it('debe multiplicar correctamente', () => {  
  expect(multiplicar(4, 5)).toBe(20);  
});
```

# TESTING DE SERVICIOS

- Servicio:

```
@Injectable()
export class UserService {

  getUsername() {
    return 'Juan';
  }

}
```

- Test:

```
it('debe devolver el nombre del usuario', () => {

  const service = new UserService();

  expect(service.getUsername()).toBe('Juan');

});
```



# TESTING DE COMPONENTES

- Componente:

```
@Component({  
  template: '<h1>{{ title }}</h1>'  
})  
export class AppComponent {  
  title = 'Angular Testing';  
}
```

- Test:

```
it('debe mostrar el título', () => {  
  
  const fixture = TestBed.createComponent(AppComponent);  
  
  fixture.detectChanges();  
  
  const h1 =  
    fixture.nativeElement.querySelector('h1');  
  
  expect(h1.textContent)  
    .toContain('Angular Testing');  
  
});
```

# TESTING DE INTERACCIÓN DE USUARIO

- Componente:

```
<button (click)="incrementar()">
  Incrementar
</button>

<p>{{ contador }}</p>
```

- Test:

```
it('debe incrementar el contador al pulsar', () => {

  const fixture =
    TestBed.createComponent(CounterComponent);

  fixture.detectChanges();

  const button =
    fixture.nativeElement.querySelector('button');

  button.click();

  fixture.detectChanges();

  expect(fixture.componentInstance.contador)
    .toBe(1);

});
```

# Angular 20



Completa nuestra encuesta  
de satisfacción a través del QR

Curso Angular 20 (6-10 julio)



GRACIAS

indra